Lossy Compression and Error Correction: Final Problem Set

```
pip install numpy scipy pillow galois
```

Useful values: $\cos\frac{\pi}{8}\approx 0.92388$, $\cos\frac{3\pi}{8}\approx 0.38268$, $\frac{1}{\sqrt{2}}\approx 0.70711$, $\log_{10}2\approx 0.30103$.

JPEG Compression

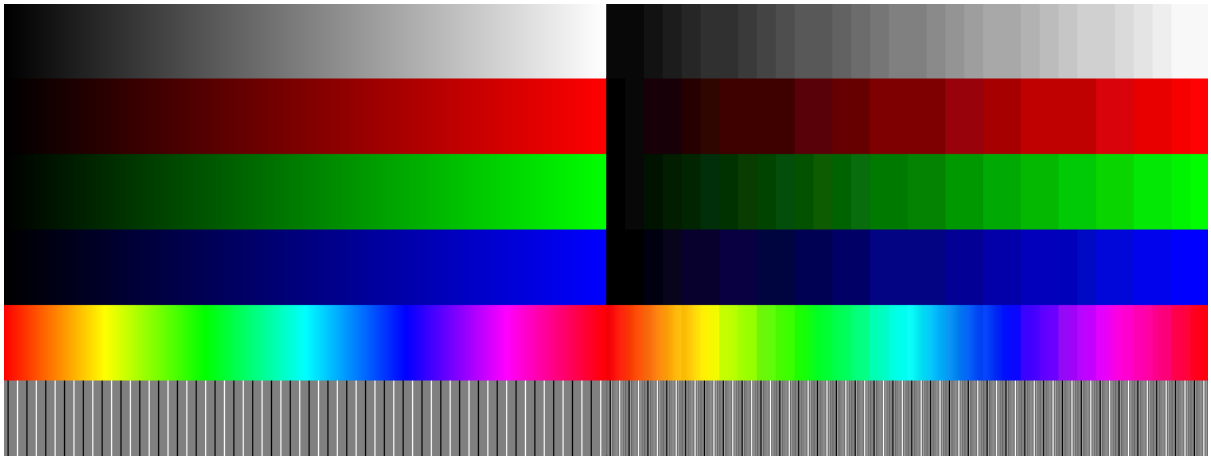
Problem 1 (DCT by hand) (on paper): Use the orthonormal DCT with $N=4$ from the lecture [5. Lossy Compression: Images and Audio]({{ '/lectures/lossy_images_and_audio/' | relative_url }}) (section "A Numerical Example: DCT with $N=4$"): $y=Cx$, where the rows of the matrix C are (a,a,a,a), $(b,c,-c,-b)$, $(a,-a,-a,a)$, $(c,-b,b,-c)$ and $a=\frac{1}{2}$, $b=\frac{1}{\sqrt{2}}\cos\frac{\pi}{8}$, $c=\frac{1}{\sqrt{2}}\cos\frac{3\pi}{8}$.

- **(a)** Compute $y=Cx$ for the signal $x=(4,8,8,4)$. Check Parseval's identity $\sum x_n^2=\sum y_k^2$.
- **(b)** Why are the coefficients y_1 and y_3 zero? State a general claim about symmetric signals ($x_n=x_{N-1-n}$) and prove it using the symmetry of the basis vectors.
- **(c)** We quantize all coefficients with the same step $Q=10$: $\hat{y}_k=Q\cdot round(y_k/Q)$. Find the reconstructed signal $x'=C^T\hat{y}$ and the squared error $\sum(x_n-x'_n)^2$. Check that it is equal to $\sum(y_k-\hat{y}_k)^2$.
- **(d)** In a 4×4 block A, all four rows are $(4,8,8,4)$. Without multiplying out all 16 entries, find $B=CAC^T$. *Hint:* first transform the columns (each column is constant), then the rows.
- **(e)** Before the DCT, JPEG subtracts 128 from every pixel of an 8×8 block. Which coefficient $B_{u,v}$ changes because of this, and by how much? How is $B_{0,0}$ related to the average luma of the block?

Problem 2 (Color spaces) (on paper): An orange pixel has $(R,G,B)=(255,128,0)$.

- **(a)** Compute Y, Cb, Cr with the JFIF formulas ($Y=0.299R+0.587G+0.114B$, $Cb=128+\frac{B-Y}{1.772}$, $Cr=128+\frac{R-Y}{1.402}$) and round them to integers.
- **(b)** Reconstruct (R,G,B) from the rounded values with the inverse formulas and round. Do you get the original color? In which step is information lost?
- **(c)** Convert the same color to the YIQ color space ($R,G,B\in[0;1]$, i.e., the values divided by 255; the matrix M is in the lecture). Which of the components I, Q is close to zero? Relate this to the claim in the lecture that the I axis corresponds to the transition from orange to blue.
- **(d)** What happens if, for a color with the same Y, the signs of I and Q are changed, i.e., $(Y,I,Q)\to(Y,-I,-Q)$? Find the corresponding (R,G,B) with M^{-1} and explain why not every YIQ triple is a displayable RGB color.
- **(e)** Prove that for gray colors ($R=G=B$) always $Cb=Cr=128$ and $I=Q=0$.
- **(f)** An image has 1000×1000 pixels. How many bytes do the uncompressed Y, Cb, Cr planes take with 4:4:4, 4:2:2 and 4:2:0 chroma subsampling? Why are Cb and Cr subsampled, but not Y?

Problem 3 (Reproduce the damage) (on a computer): Use the program jpeg_lab.py from the lab Breaking JPEG on Purpose (the parameters are described in the table "How the program is organized" of the lab, or run `python jpeg_lab.py --help`).



The image to the right was produced by `jpeg_lab.py`.
The original is a synthetic image gradient-bands.png (512×384, drawn by make_gradient_bands.py) contains a grayscale gradient, R, G, B gradients, a hue ramp and thin vertical lines. The image target-d.png was produced from it by changing two parameters ($PSNR = 29.20$ dB, size 1.4 KiB). Find them. Why do the smooth gradients turn into "staircases", and what does the width of a stair step tell you?
Create your own synthetic image (for example, with a diagonal gradient, a circle or text) and find a combination of parameters that causes the most visible damage in it while the estimated size is not smaller than the size of the default run. Briefly describe why exactly this image is "difficult" for JPEG.

Other Image, Audio and Video Encodings

Problem 4 (Animated image) (on a computer): In the lecture, an animated AVIF file was created with the script [animated_avif.py]({{ '/lectures/lossy_images_and_audio/figs/animated_avif.py' | relative_url }}). Create a similar animation with your own scene.

- **(a)** Write a Python program (Pillow and NumPy) that draws a 2-second animation at 25 frames per second, 320×180 pixels, showing a moving object on a plain background: for example, a clock hand that turns by 360°, or a band with a color gradient that slides across the screen. The animation must be a seamless loop (the last frame must connect to the first one). How should the speed be chosen to achieve this?
- **(b)** Save the same sequence of frames in three formats: animated GIF, animated WebP and animated AVIF (`Image.save(..., save_all=True, append_images=..., duration=..., loop=0)`). Compare the file sizes with the total size of the uncompressed frames ($50 \cdot 320 \cdot 180 \cdot 3$ bytes).
- **(c)** GIF can use at most 256 colors in a frame. How does this show in a color gradient? What does Pillow do when it converts an RGB frame into a palette frame (compare `Image.quantize(..., dither=Image.Dither.NONE)` and `dither=Image.Dither.FLOYDSTEINBERG`)?
- **(d)** Add small random noise to the background (independently for every pixel and every frame, for example, ± 8). How do the sizes of all three files change? Explain why noise hurts both intra prediction within a frame and prediction from the previous frame (see P-frames in the video lecture).
- **(e)** Repeat (d) with *one* noise pattern for all frames (the noise "sticks" to the background). How do the file sizes change now compared to (d)? Which format gains the most, and why?

Problem 5 (Sound in the frequency domain) (on a computer): With NumPy, create a 2-second mono signal with the sample rate $f_s = 48\,000$ Hz: an A major chord with the tones 440 Hz (A4), $440 \cdot 2^{4/12}$ Hz (C♯5) and

$440 \cdot 2^{7/12}$ Hz (E5), each with amplitude 0.2, plus a quiet tone of 470 Hz with amplitude 0.002. Save it as a 16-bit WAV file (`scipy.io.wavfile.write`) and listen to it.

- **(a)** Compute the spectrum of the signal with `numpy.fft.rfft` and plot $|X(f)|$ in decibels ($20\log_{10}$) for the frequencies 0–2000 Hz. Find the four peaks. What is the frequency resolution (the distance between neighboring FFT frequencies in hertz) for the 2-second signal and for a 0.02-second fragment? Can the 470 Hz tone be distinguished from the 440 Hz tone in both cases?
- **(b)** "Compression in the frequency domain": split the signal into frames of $N = 1024$ samples, compute `scipy.fft.dct(..., norm="ortho")` for each frame, keep only the K coefficients with the largest absolute value (replace the others with 0) and reconstruct with `scipy.fft.idct`. For $K = 8, 16, 32, 64$, compute the signal-to-noise ratio $SNR = 10\log_{10}\dfrac{\sum x_n^2}{\sum (x_n - x_n')^2}$ and listen to the result. At which K does the quiet 470 Hz tone disappear? Is it audible in the original next to the loud 440 Hz tone (see frequency masking)?
- **(c)** In the middle of the signal (at 1.0 s), add a sharp click (one sample with the value 0.9). Repeat (b) with K equal to $1/16$ of N, for the frame lengths $N = 4096$ and $N = 256$. Plot the error $x - x'$ in time around the click. How many milliseconds before the click does the error start in both cases? Compare with the duration of pre-masking (5–20 ms) and explain why MP3 and Opus use short windows at transients.
- **(d)** Create a 5000 Hz tone with $f_s = 8000$ Hz and listen to it (set $f_s = 8000$ in the WAV file). Which frequency do you hear? Check with the FFT and explain with the Nyquist theorem.
- **(e)** *(Optional.)* Encoders do not just code "the K largest coefficients": instead, they choose a quantization step for each frequency band. Quantize the DCT coefficients with a step that grows with frequency (for example, $q_k = q_0 \cdot (1 + k/64)$), and compare the SNR and the sound with (b) at roughly the same number of nonzero coefficients.

Problem 6 (Theory questions about codecs) (on paper): For each statement, say whether it is true or false and justify it with one or two sentences (or a counterexample).

- **(a)** Quantization is the only JPEG step in which information is lost.
- **(b)** If the DCT coefficients are quantized with step Q and the rounding error is uniformly distributed, then the mean squared error in pixels is $Q^2/12$ regardless of how many coefficients become zero.
- **(c)** Doubling all quantization steps makes a JPEG file about two times smaller.
- **(d)** A PSNR of 40 dB means that the mean squared error is 100 times smaller than at a PSNR of 20 dB.
- **(e)** An AVIF decoder is about as slow as an AVIF encoder, because both perform the same steps.
- **(f)** A video with the GOP structure `IBBPBBPBB` can be watched starting from any frame without decoding the previous ones.
- **(g)** An Opus file at 32 kbit/s contains exact frequency components up to 20 kHz.
- **(h)** Changing the file extension from `.webm` to `.mkv` changes neither the container nor the codec, but many players will play the file anyway.

Hamming Codes and Tanner Graphs

Problem 7 (The Hamming code and its extension) (on paper): We use the Hamming code $[7, 4, 1]$ from the lecture [7. Error Correction: Hamming Codes]({{ '/lectures/lossy_hamming/' | relative_url }}): the string $x_1x_2x_3x_4$ is transmitted as $x_1x_2x_3y_1x_4y_2y_3$, where $y_1 = x_1 \oplus x_2 \oplus x_3$, $y_2 = x_1 \oplus x_2 \oplus x_4$, $y_3 = x_1 \oplus x_3 \oplus x_4$. The positions have the binary indices $111, 110, 101, 100, 011, 010, 001$.

- **(a)** Encode the message `1011`.

- **(b)** `1110101` is received. Compute the *syndrome* (s_1, s_2, s_3), where $s_i = 1$ if and only if the check bit y_i does not agree with its formula. Show that the syndrome, written as the three-bit number $s_1 s_2 s_3$, is exactly the binary index of the erroneous position. Correct the error.
- **(c)** *Two* last bits of the codeword from (a) were corrupted, and `1010110` was received. What will a decoder do that assumes at most one error? Which message will it output?
- **(d)** The extended Hamming code adds an eighth bit y_0 – the parity of all seven bits. Prove that any two codewords of this code differ in at least 4 positions. How can the decoder use y_0 and the syndrome to distinguish the cases "no errors", "one error (correctable)" and "two errors (only detectable)"? Apply this to the situation of (c).
- **(e)** Prove that $y_0 = x_2 \oplus x_3 \oplus x_4$ for every codeword.

Problem 8 (A Tanner graph and erasures) (on paper; (e) on a computer): A code with 8 bits $b_1, \dots, b_8$ is defined by four checks:

$$
\begin{aligned}
p_1 &: b_1 \oplus b_2 \oplus b_3 \oplus b_5 = 0, \\
p_2 &: b_1 \oplus b_2 \oplus b_4 \oplus b_6 = 0, \\
p_3 &: b_1 \oplus b_3 \oplus b_4 \oplus b_7 = 0, \\
p_4 &: b_2 \oplus b_3 \oplus b_4 \oplus b_8 = 0.
\end{aligned}
$$

- **(a)** Write down the parity-check matrix H and draw the Tanner graph. Prove that $b_1, \dots, b_4$ can be chosen freely, while $b_5, \dots, b_8$ are uniquely determined by them. Encode $b_1 b_2 b_3 b_4 = $ `1011`. Compare with Problem 7: which code is this?
- **(b)** In an erasure channel, the bits b_1, b_2, b_5 of the codeword from (a) were lost. Recover them with the peeling decoder, stating the check used in each step.
- **(c)** The bits b_1, b_2, b_3, b_4 were lost. Show that the peeling decoder "gets stuck" already in the first step. Can the lost bits nevertheless be recovered uniquely by solving a system of four equations modulo 2? Recover them.
- **(d)** Find a set of four erasures that *cannot* be recovered uniquely by any algorithm. How is it related to the codewords with exactly four ones? Why can any three erasures always be recovered?
- **(e)** *(On a computer.)* Write a function `peel(H, received)` that recovers erasures (marked with `None`) by peeling. Generate a random LDPC code with $n = 1200$ bits, in which every bit is in 3 checks and every check has 6 bits (for example, by shuffling the $3n$ "ends" of the edges in random order). For each $\varepsilon \in \{0.30, 0.35, 0.40, 0.45, 0.50\}$, erase every bit with probability ε, 100 times, and count how many times peeling recovers everything. Explain why it is enough to test only the all-zero codeword. At which ε does decoding suddenly start to fail? (The theoretical threshold for such codes is $\varepsilon \approx 0.429$; the largest possible one is $1 - k/n = 0.5$.)

Reed–Solomon Codes and Finite Fields

Problem 9 (A Reed–Solomon code over GF(7)) (on paper): Over the field $\mathrm{GF}(7) = \{0, 1, \dots, 6\}$ (arithmetic modulo 7), a message block has $k = 2$ numbers a_1, a_2, the message polynomial is $f(x) = a_1 x + a_2$, and $s = 5$ values $f(0), f(1), \dots, f(4)$ are sent. The notation is the same as in the lecture [8. Error Correction: Reed–Solomon Codes]({{ '/lectures/lossy_reed_solomon/' | relative_url }}).

- **(a)** Write the table of inverses: a^{-1} for every $a \in \{1, \dots, 6\}$.
- **(b)** Encode the message $3, 5$.
- **(c)** How many lost values (erasures) and how many erroneous values does this code correct with certainty? What is the largest s that can be used over $\mathrm{GF}(7)$, and how many errors could be corrected then?

- **(d)** $r = (*, 1, *, *, 3)$ is received, where * is a lost value. Recover the message with Lagrange interpolation.
- **(e)** $r = (5, 1, 6, 0, 3)$ is received; one value may be wrong, but it is not known which one. With the Berlekamp–Welch method ($Y(x) = x + y_0$, $Z(x) = z_2 x^2 + z_1 x + z_0$), find the position of the error and the message.
- **(f)** Check your answer in another way: find a line that passes through at least four of the five points (i, r_i). Why is such an exhaustive search impractical for larger k and s?

Problem 10 (The field GF(8) and Reed–Solomon codes for bytes) (on paper (a)–(c); on a computer (d)–(f)): The field $\mathrm{GF}(8)$ consists of polynomials with coefficients in $\{0, 1\}$ and degree at most 2, and multiplication is done modulo $p(x) = x^3 + x + 1$ (see the tables in the lecture). We also write the element $b_2 x^2 + b_1 x + b_0$ as the three-bit string $b_2 b_1 b_0$ or as a number ($x^2 + 1 = $ `101` $= 5$). Let $\alpha = x$.

- **(a)** Compute $\alpha^0, \alpha^1, \ldots, \alpha^7$ as three-bit strings. Make sure that $\alpha^0, \ldots, \alpha^6$ are all seven nonzero elements (i.e., α is a *primitive* element).
- **(b)** Using the table from (a) as a "table of logarithms", compute $(x^2 + 1) \cdot (x + 1)$ and $(x^2 + x)^{-1}$. Check with the multiplication table in the lecture. Why is addition in this field a bitwise XOR?
- **(c)** A Reed–Solomon code over $\mathrm{GF}(8)$ with $k = 2$: $f(x) = a_1 x + a_2$. Encode the message $a_1 = \alpha$, $a_2 = 1$ by computing the values of f at the points $0, 1, \alpha, \alpha^2$. How many bits are there in the message and how many in the codeword?
- **(d)** In the library `galois`, a field is created with `GF = galois.GF(2**3)`; its elements are NumPy arrays that support `+`, `*`, `**` and `np.linalg.solve`. Check the computations of (a)–(c) with `galois`. Then write the functions `rs_encode(msg, s)` and `berlekamp_welch(r, k)` in the notation of the lecture (values at the points $0, 1, \alpha, \ldots$) and test them over $\mathrm{GF}(2^8)$ with $k = 8$, $s = 16$ and randomly corrupted values. How many errors can be corrected, and what happens if there is one error more?
- **(e)** The class `galois.ReedSolomon(255, 223)` implements the code over $\mathrm{GF}(2^8)$ that is used in CD and space communication standards (each symbol is a byte; the code is *systematic* – the message bytes stay unchanged, and 32 check bytes are appended to them; this differs from the version in the lecture, but the underlying linear algebra is the same). Encode a text message with it, corrupt 16 consecutive bytes (128 bits) and decode with `rs.decode(..., errors=True)`. Repeat with 17 corrupted bytes. What does the decoder return?
- **(f)** Compare the result of (e) with the Hamming code $[7, 4, 1]$, which has proportionally even more redundant bits (3 out of 7, compared to 32 out of 255): the $223 \cdot 8$ message bits are split into blocks of 4 bits, and each block is encoded separately. How many of the 128 consecutive corrupted bits could the Hamming code correct? Why are Reed–Solomon codes suitable for burst errors, for example, scratches on a compact disc?